Speaker Diarization - Code Breakdown and Problem Summary

Lauren Hendley

March 18, 2026

Abstract

This document covers the problem and what the program does, how it works, and how to run it, including all dependencies.

1 Problem definition

Speaker diarization is a process that takes an audio file and condenses it into distinct, labeled speaker turns. Essentially, it determines 'who spoke when'. The applications of this extend from interviews in which patient responses must be accurately tracked to meetings and interviews. The typical output of speaker diarization is of the following form:

[SPEAKER0] : Words...
[SPEAKER1] : More words...

This problem is applicable, more notably in this situation, for gathering results from spoken interviews from patients. This extends further into gathering information from audio resources exponentially faster.
It is a process that involves the following:

- Speech activity detection
- Feature extraction
- Speaker representation
- Segmentation
- Clustering/classification

2 Common issues

Speaker diarization is made more difficult with common issues including:

- Environmental noise
- Unknown number of speakers
- Overlapping speech (traditionally the most difficult, as most older models assume by default that only one individual speaks at a time)
- Speaker variability
- ASR errors (if transcript-based correction is employed)

3 Literature review

In order to fully grasp the problem and the current market of solutions, a small literature review was completed.

3.1 Traditional models

Typically follow a modular design.
Audio is segmented into speech regions (sections of an audio where speech is detected). These are transformed into features and learned speaker embeddings which are then grouped into speaker clusters (assigning the words to a person) [1].
Traditional models include: X-Vector, hidden Markov, Gaussian mixture, and MFCCs.

3.1.1 X-Vector

Another common method in speaker diarization is the x-vector approach which extracts speakers using a time-delay neural network [1].
This is a pipeline that follows the modular design of traditional models. Speaker embeddings are extracted from speech segments, and are then clustered. The methods of clustering include: agglomerative hierarchical clustering, spectral clustering, k-means, and Gaussian mixture model-based clustering [2].

- Agglomerative hierarchical clustering:
- Spectral clustering:
- K-means:
- Gaussian mixture model-based clustering:

Segmentation and clustering generally function as strong baselines when the speech is non-overlapping and relatively clean, but are less reliable for distorted or overlapped audio.

3.2 Modern models

3.2.1 End-to-end neural diarization (EEND)

Recently, there has been a shift towards end-to-end-diarization models (EENDs) which integrate speaker recognition in the transcription [1]. The diarization error rate (DER) is optimized over Bidirectional Long Short-Term Memory models (BLSTMs) [1].
EEND models and similar neural architectures model segmentation and speaker assignment jointly rather then in a pipeline. This is particularly useful with audio files that have more overlapping speech.
EEND is limited with the amount of data required for training, which can be curved with the incorporation of self-supervised learning (SSL) [4]. This was not employed in this solution or code, but it does demonstrate substantial improvements over the traditional models with Pyannote and are a promising direction for speaker diarization [4].

3.2.2 Large language models (LLM)

The results of this literature review showed that the highest presenting accuracy is achieved with LLM-backed speaker diarization models.
The accuracy of these models can be affected by quality, speaker behaviour variability, environmental noise, and overlapping speech [1].
LLMs are employed as a post-processing filter. They take a traditional ASR output and use context to correct speaker assignments.
LLMs are limited as performance degrades when the model is applied to transcripts produced by a different ASR system than what was used in fine-tuning [1]. To reduce this, multiple ASR-specific models were accumulated into an ensemble model, which is far more generalizable. Effectiveness of LLMs depend heavily on the quality and consistency of the transcript input.

3.2.3 Speaker counting and separation

This framework is for multichannel multispeaker conversations and implements speaker counting, diarization, and separation. In order to perform this, they use a guided target speaker extraction (G-TSE) and a guided source separation (GSS). These improve performance in difficult environments.

4 Methodology and datasets

In order to test accuracy, two separate programs were created.
The diarization pipeline is implemented using a WhisperX and Pyannote collaboration, and the LLM model is sourced from the DiarizationLM library (find more information on DiarizationLM here: https://github.com/google/speaker-id/blob/master/DiarizationLM/README.md).

Note that all datasets and processes were found through HuggingFace except for the rttm files for testing. These were found on the following github repo: https://github.com/joonson/voxconverse.

Note that if issues pop up with version control between numpy, torchvision, etc, run the following prompt to align versions:
!pip install "numpy¡2.0" datasets==2.21.0 torchvision==0.20.0 torchaudio==2.5.0 torch==2.5.0

4.1 Simple diarization

Listing 1: Simple Diarization (No LLM)

```python
## Speaker diarization implementation using WhisperX and Pyannote
## Author: Lauren Hendley

## Install dependencies:
##   pip install whisperx datasets pyannote.metrics ffmpeg-python soundfile
##   !apt install -y ffmpeg  (Linux/Colab)

import whisperx
import torch
import subprocess
import numpy as np
import getpass
import os
from datasets import load_dataset, Audio

from pyannote.metrics.diarization import DiarizationErrorRate
from pyannote.database.util import load_rttm

from huggingface_hub import login


# ----------------------------
# INPUT FUNCTIONS
# ----------------------------


def get_file():
    """
    Getting the file

    Returns:
        dict: sample of VoxConverse test
    """
    print("Getting the file...")

    ds = load_dataset("diarizers-community/voxconverse", # VoxConverse dataset
                      split = "test", # 16-sample test split
                      streaming = True) # Avoids downloading full dataset

    ds = ds.cast_column("audio", Audio(sampling_rate = 16000)) # Resample to 16kHz (required for WhisperX)

```

```python
    return next(iter(ds))


# Tries locally in colab first, then requests if fails (either not in colab or token isn't stored in colab)
try:
    from google.colab import userdata
    def get_token():
        """
        Getting the user's token

        Returns:
            String: user's token
        """
        try:
            return userdata.get('HF_TOKEN')
        except:
            return getpass.getpass("Enter HuggingFace token: ")
except Exception:
    def get_token():
        """
        Getting the user's token

        Returns:
            String: user's token
        """
        return getpass.getpass("Enter HuggingFace token: ")


def clone_voxconverse_rttms(dest="voxconverse"):
    """
    Clone the VoxConverse GitHub repo to access its RTTM annotation files.
    Skips cloning if the repo already exists locally.

    Args:
        dest (str): local directory name to clone into

    Returns:
        str: path to the test/ RTTM directory inside the cloned repo
    """
    if not os.path.exists(dest):
        print("Cloning VoxConverse RTTM annotations...")
        subprocess.run(
            ["git", "clone", "--depth", "1",
             "https://github.com/joonson/voxconverse.git", dest],
            check = True
        )
    else:
        print(f"VoxConverse repo already exists at '{dest}', skipping clone.")

    return os.path.join(dest, "test")


def get_rttm_annotations(rttm_dir, file_id):
    """
    Load the reference RTTMs.

    Args:
        rttm_dir (str): path to the cloned repo's test/ directory
        file_id (str): the sample identifier (e.g. 'abjxc')

    Returns:
```

```python
        Annotation: pyannote reference annotation for DER evaluation
    """
    rttm_path = os.path.join(rttm_dir, f"{file_id}.rttm")

    if not os.path.exists(rttm_path):
        raise FileNotFoundError(f"RTTM file not found: {rttm_path}")

    # load_rttm returns a dict of {uri: Annotation}
    ref_annotations = load_rttm(rttm_path)

    # There is one entry per file; return the single annotation
    return list(ref_annotations.values())[0]


# ----------------------------
# TRANSCRIBE AND ALIGN
# ----------------------------


def load_transcribe_align(audio, device):
    """
    Load whisper model, transcribe and align timestamps

    Args:
        audio (array): audio dervied from file
        device (String): 'cuda' or 'cpu

    Returns:
        dict: segments with timestamps
    """
    print("Starting transcribe...")

    compute_type = "float16" if device == "cuda" else "int8" # Changing the compute type if using GPU
    # float16 used on GPU (faster inference)
    # int8 used on CPU (reduce memory usage, improve speed)


    ## Load model (WhisperX)
    model = whisperx.load_model("small", # Can be upgraded to medium or large to improve accuracy
                                device,
                                compute_type = compute_type)

    ## Transcribe (into segments with timestamps)
    res = model.transcribe(audio, batch_size = 16)


    ## Align words to audio (using alignment model)
    model_a, metadata = whisperx.load_align_model(
        language_code = "en", # Strictly English
        device = device
    )

    res = whisperx.align(res["segments"], ## Transcribed audio
                         model_a,
                         metadata,
                         audio,
                         device)

    return res

```

```python

# ----------------------------
# DIARIZATION
# ----------------------------


def diarization(token, res, audio, device):
    """ Perform diarization on the audio
    Args:
        token (string): user's token
        res (dict): aligned transcription segments
        audio (np.ndarray): audio array
        device (string): device

    Returns:
        dict: segments with speaker labels
        Annotation: Pyannote annotation
    """
    print("Starting diarization...")

    login(token = token)

    diarize_model = whisperx.diarize.DiarizationPipeline(
        device = device
    )

    ## Diarization pipeline (limits to 2-6 speakers)
    diarization_res = diarize_model(audio, min_speakers = 2, max_speakers = 6)

    ## Mapping speakers onto word segments
    final = whisperx.assign_word_speakers(diarization_res, res)

    return final, diarization_res


# ----------------------------
# MAIN
# ----------------------------


if __name__ == "__main__":
    torch.manual_seed(42)

    device = "cuda" if torch.cuda.is_available() else "cpu" # Choosing device based on GPU use or not

    rttm_dir = clone_voxconverse_rttms()

    ## Get the file, token, and audio
    fn = get_file()
    audio = fn["audio"]["array"].astype(np.float32) # Extracts the audio as an NP array (float32) - required by WhisperX
    file_id = fn["audio"]["path"].split("/")[-1].replace(".wav", "")
    token = get_token()

    ## Transcribing and aligning
    res = load_transcribe_align(audio, device)

    ## Diarization process + assigning speakers
    final, diarization_res = diarization(token, res, audio, device)

    ## Outputs the results (speaker-labelled)
```

```
    print("Printing segments...")
    for seg in final["segments"]:
        speaker = seg.get("speaker", "UNKNOWN")

        print(f"[{speaker}] : {seg['text']}")

    ref_ann = get_rttm_annotations(rttm_dir, file_id)

    ## Calculating the error (against annotation)
    metric = DiarizationErrorRate()
    metric(ref_ann, diarization_res)
    der = abs(metric)
    print(f"\nDiarization Error Rate: {der:.3f}")
```

4.2 LLM-backed diarization

Listing 2: LLM-backed diarization

```
## Speaker diarization implementation using WhisperX and Pyannote
## Author: Lauren Hendley

## Install dependencies:
##   pip install whisperx datasets pyannote.metrics ffmpeg-python soundfile
##   !apt install -y ffmpeg  (Linux/Colab)

import whisperx
import torch
import subprocess
import numpy as np
import getpass
import os
from datasets import load_dataset, Audio

from pyannote.metrics.diarization import DiarizationErrorRate
from pyannote.database.util import load_rttm

from huggingface_hub import login


# ----------------------------
# INPUT FUNCTIONS
# ----------------------------


def get_file():
    """
    Getting the file

    Returns:
        dict: sample of VoxConverse test
    """
    print("Getting the file...")

    ds = load_dataset("diarizers-community/voxconverse", # VoxConverse dataset
                      split = "test", # 16-sample test split
                      streaming = True) # Avoids downloading full dataset

    ds = ds.cast_column("audio", Audio(sampling_rate = 16000)) # Resample to
        16kHz (required for WhisperX)

    return next(iter(ds))
```

```python


# Tries locally in colab first, then requests if fails (either not in colab or token isn't stored in colab)
try:
    from google.colab import userdata
    def get_token():
        """
        Getting the user's token

        Returns:
            String: user's token
        """
        try:
            return userdata.get('HF_TOKEN')
        except:
            return getpass.getpass("Enter HuggingFace token: ")
except Exception:
    def get_token():
        """
        Getting the user's token

        Returns:
            String: user's token
        """
        return getpass.getpass("Enter HuggingFace token: ")


def clone_voxconverse_rttms(dest="voxconverse"):
    """
    Clone the VoxConverse GitHub repo to access its RTTM annotation files.
    Skips cloning if the repo already exists locally.

    Args:
        dest (str): local directory name to clone into

    Returns:
        str: path to the test/ RTTM directory inside the cloned repo
    """
    if not os.path.exists(dest):
        print("Cloning VoxConverse RTTM annotations...")
        subprocess.run(
            ["git", "clone", "--depth", "1",
             "https://github.com/joonson/voxconverse.git", dest],
            check = True
        )
    else:
        print(f"VoxConverse repo already exists at '{dest}', skipping clone.")

    return os.path.join(dest, "test")


def get_rttm_annotations(rttm_dir, file_id):
    """
    Load the reference RTTMs.

    Args:
        rttm_dir (str): path to the cloned repo's test/ directory
        file_id (str): the sample identifier (e.g. 'abjxc')

    Returns:
        Annotation: pyannote reference annotation for DER evaluation
```

```python
    """
    rttm_path = os.path.join(rttm_dir, f"{file_id}.rttm")

    if not os.path.exists(rttm_path):
        raise FileNotFoundError(f"RTTM file not found: {rttm_path}")

    # load_rttm returns a dict of {uri: Annotation}
    ref_annotations = load_rttm(rttm_path)

    # There is one entry per file; return the single annotation
    return list(ref_annotations.values())[0]


# ----------------------------
# TRANSCRIBE AND ALIGN
# ----------------------------


def load_transcribe_align(audio, device):
    """
    Load whisper model, transcribe and align timestamps

    Args:
        audio (array): audio dervied from file
        device (String): 'cuda' or 'cpu

    Returns:
        dict: segments with timestamps
    """
    print("Starting transcribe...")

    compute_type = "float16" if device == "cuda" else "int8" # Changing the compute type if using GPU
    # float16 used on GPU (faster inference)
    # int8 used on CPU (reduce memory usage, improve speed)


    ## Load model (WhisperX)
    model = whisperx.load_model("small", # Can be upgraded to medium or large to improve accuracy
                                device,
                                compute_type = compute_type)

    ## Transcribe (into segments with timestamps)
    res = model.transcribe(audio, batch_size = 16)


    ## Align words to audio (using alignment model)
    model_a, metadata = whisperx.load_align_model(
        language_code = "en", # Strictly English
        device = device
    )

    res = whisperx.align(res["segments"], ## Transcribed audio
                         model_a,
                         metadata,
                         audio,
                         device)

    return res


```

```python
# ----------------------------
# DIARIZATION
# ----------------------------


def diarization(token, res, audio, device):
    """ Perform diarization on the audio
    Args:
        token (string): user's token
        res (dict): aligned transcription segments
        audio (np.ndarray): audio array
        device (string): device

    Returns:
        dict: segments with speaker labels
        Annotation: Pyannote annotation
    """
    print("Starting diarization...")

    login(token = token)

    diarize_model = whisperx.diarize.DiarizationPipeline(
        device = device
    )

    ## Diarization pipeline (limits to 2-6 speakers)
    diarization_res = diarize_model(audio, min_speakers = 2, max_speakers = 6)

    ## Mapping speakers onto word segments
    final = whisperx.assign_word_speakers(diarization_res, res)

    return final, diarization_res


# ----------------------------
# MAIN
# ----------------------------


if __name__ == "__main__":
    torch.manual_seed(42)

    device = "cuda" if torch.cuda.is_available() else "cpu" # Choosing device based on GPU use or not

    rttm_dir = clone_voxconverse_rttms()

    ## Get the file, token, and audio
    fn = get_file()
    audio = fn["audio"]["array"].astype(np.float32) # Extracts the audio as an NP array (float32) - required by WhisperX
    file_id = fn["audio"]["path"].split("/")[-1].replace(".wav", "")
    token = get_token()

    ## Transcribing and aligning
    res = load_transcribe_align(audio, device)

    ## Diarization process + assigning speakers
    final, diarization_res = diarization(token, res, audio, device)

    ## Outputs the results (speaker-labelled)
    print("Printing segments...")
```

```
    for seg in final["segments"]:
        speaker = seg.get("speaker", "UNKNOWN")

        print(f"[{speaker}] : {seg['text']}")

    ref_ann = get_rttm_annotations(rttm_dir, file_id)

    ## Calculating the error (against annotation)
    metric = DiarizationErrorRate()
    metric(ref_ann, diarization_res)
    der = abs(metric)
    print(f"\nDiarization Error Rate: {der:.3f}")
```

5 Comparison of accuracy

Accuracy comparison here

5.1 Simple dataset

The simple dataset used is the VoxConverse dataset. An open-source dataset containing short spoken sentences of multiple languages. In order to compare the results of the diarization, the rttm (Rich Transcription Time Marked) files provided were used.

5.2 Complicated dataset

Unable to run due to machine constraints. Further analysis required.

Glossary

ASR Automatic Speech Recognition. A system that converts spoken audio into text. In some diarization systems, ASR transcripts are also used to help assign speaker labels. 2

BLSTM Bidirectional Long Short-Term Memory. A type of recurrent neural network that processes information in both forward and backward time directions, making it useful for analyzing speech sequences. 2

DER Diarization Error Rate. A common metric used to measure speaker diarization accuracy. It reflects errors in speaker assignment, missed speech, and false detections. Lower DER means better performance. 2

EEND End-to-End Neural Diarization. A diarization method that uses a single neural network to directly determine who spoke when, instead of separating the task into multiple stages such as segmentation and clustering. 2

G-TSE Guided Target Speaker Extraction. A method that uses guidance information to isolate the speech of a specific speaker from a mixed audio signal. 2

GSS Guided Source Separation. A method for separating overlapping speech or multiple sound sources by using prior information to improve the separation process. 2

LLM Large Language Model. A neural network trained on large amounts of text data that can understand and generate language. In speaker diarization, it can be used to improve or correct speaker labels using transcript context. 2

MFCC Mel-Frequency Cepstral Coefficients. Audio features that summarize the important frequency characteristics of speech and are commonly used in speech and speaker recognition. 2

SSL Self-Supervised Learning. A machine learning approach where a model learns useful representations from unlabeled data. In speech processing, SSL models can improve diarization performance when labeled training data is limited. 2

X-Vector A fixed-length speaker embedding extracted from a short segment of speech. It represents speaker characteristics and is commonly used in diarization systems for clustering and speaker comparison. 2

References

[1] G. Efstathiadis, V. Yadav, and A. Abbas, "LLM-based speaker diarization correction: A generalizable approach," *Speech Communication*, vol. 170, p. 103224, May 2025. doi: https://doi.org/10.1016/j.specom.2025.103224.

[2] A. Mitrofanov *et al.*, "Accurate speaker counting, diarization and separation for advanced recognition of multichannel multispeaker conversations," *Computer Speech & Language*, vol. 92, p. 101780, Jun. 2025. doi: https://doi.org/10.1016/j.csl.2025.101780.

[3] D. O'Shaughnessy, "Speaker Diarization: A Review of Objectives and Methods," *Applied Sciences*, vol. 15, no. 4, p. 2002, Feb. 2025. doi: https://doi.org/10.3390/app15042002.

[4] J. Han, F. Landini, J. Rohdin, A. Silnova, M. Diez, and L. Burget, "Leveraging Self-Supervised Learning for Speaker Diarization," *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, Apr. 2025. doi: https://doi.org/10.1109/icassp49660.2025.10889475.

[5] G. Chen, A. Alsharef, A. Ovid, A. Albert, and E. Jaselskis, "Meet2Mitigate: An LLM-powered framework for real-time issue identification and mitigation from construction meeting discourse," *Advanced Engineering Informatics*, vol. 64, p. 103068, Dec. 2024. doi: https://doi.org/10.1016/j.aei.2024.103068.